

CSCE 3110
Data Structures
& Algorithm Analysis

Algorithm Analysis I

Reading: *Weiss, chap.2*



Problem Solving: Main Steps

1. Problem definition
2. Algorithm design / Algorithm specification
3. Algorithm analysis
4. Implementation
5. Testing
6. [Maintenance]

1. Problem Definition

- What is the task to be accomplished?
  - Calculate the average of the grades for a given student
  - Understand the talks given out by politicians and translate them in Chinese
- What are the time / space / speed / performance requirements ?



2. Algorithm Design / Specifications

- Algorithm: Finite set of instructions that, if followed, accomplishes a particular task.
- Describe: in natural language / pseudo-code / diagrams / etc.
- Criteria to follow:
  - Input: Zero or more quantities (externally produced)
  - Output: One or more quantities
  - Definiteness: Clarity, precision of each instruction
  - Finiteness: The algorithm has to stop after a finite (may be very large) number of steps
  - Effectiveness: Each instruction has to be basic enough and feasible
    - ~~Understand speech~~
    - ~~Translate to Chinese~~



Computer Algorithms

An **algorithm** is a procedure (a finite set of well-defined instructions) for accomplishing some tasks which,

- given an *initial state*
- terminate in a *defined end-state*

The **computational complexity** and **efficient implementation** of the algorithm are important in computing, and this depends on suitable data structures.



4,5,6: Implementation, Testing, Maintainance

- Implementation
  - Decide on the programming language to use
    - C, C++, Lisp, Java, Perl, Prolog, assembly, etc. , etc.
  - Write clean, well documented code
- Test, test, test
- Integrate feedback from users, fix bugs, ensure compatibility across different versions → Maintenance



3. Algorithm Analysis

- Space complexity
  - How much space is required
- Time complexity
  - How much time does it take to run the algorithm

- Often, we deal with estimates!



Space Complexity

- Space complexity = The amount of memory required by an algorithm to run to completion
  - [Core dumps = the most often encountered cause is “dangling pointers”]
- Some algorithms may be more efficient if data completely loaded into memory
  - Need to look also at system limitations
  - E.g. Classify 2GB of text in various categories [politics, tourism, sport, natural disasters, etc.] – can I afford to load the entire collection?



Space Complexity (cont'd)

1. Fixed part: The size required to store certain data/variables, that is independent of the size of the problem:
   - e.g. name of the data collection
   - same size for classifying 2GB or 1MB of texts

2. Variable part: Space needed by variables, whose size is dependent on the size of the problem:
   - e.g. actual text
   - load 2GB of text VS. load 1MB of text



Space Complexity (cont'd)

- S(P) = c + S(instance characteristics)
  - c = constant
- Example:

```
float summation(const float (&a)[10], int n )
{
   float s = 0;
   int i;
   for(i = 0; i<n; i++) {
     s+= a[i];
   }
   return s;
}
```

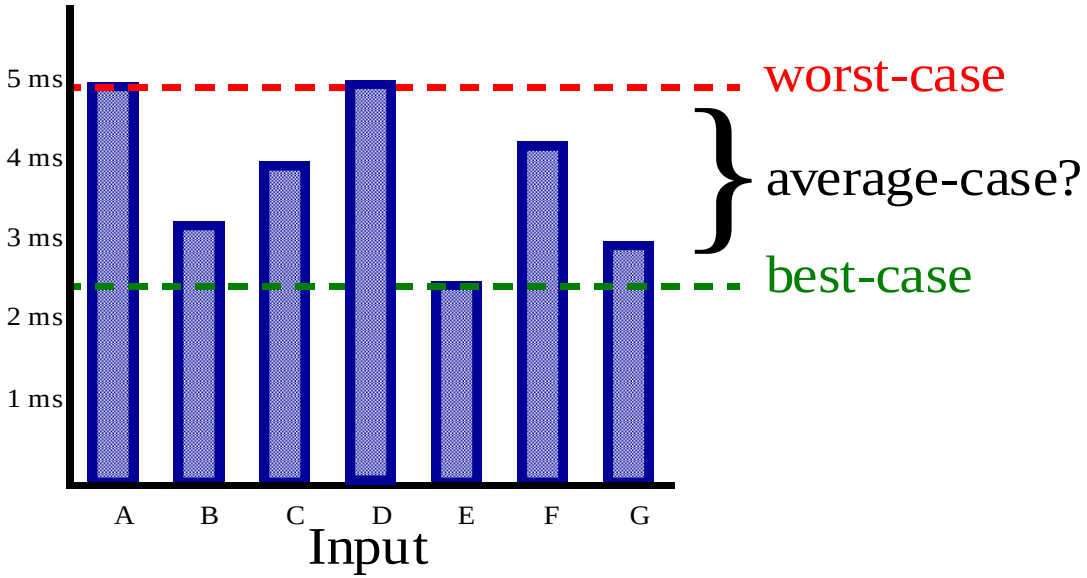
Space? one for n, one for a [passed by reference!], one for i → constant space!



Time Complexity

- Often more important than space complexity
  - space available (for computer programs!) tends to be larger and larger
  - time is still a problem for all of us
- 3-4GHz processors on the market
  - still …
  - researchers estimate that the computation of various transformations for 1 single DNA chain for one single protein on 1 TerraHZ computer would take about 1 year to run to completion
- Algorithms running time is an important issue

Running time



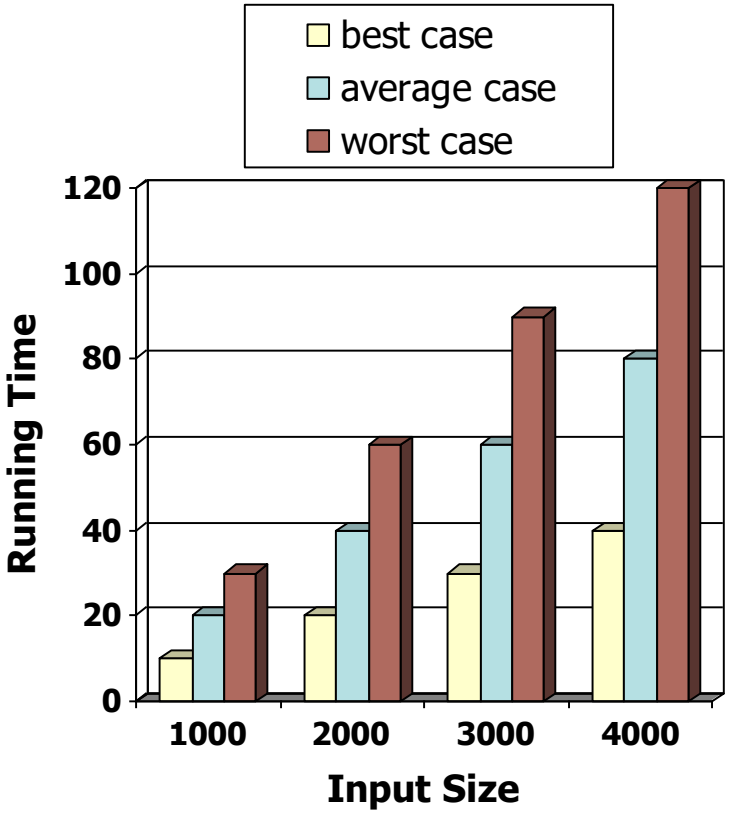
Suppose the program includes an *if-then* statement that may execute or not: → variable running time

Typically algorithms are measured by their ***worst case***



Running Time

- The running time of an algorithm varies with the inputs, and typically grows with the size of the inputs.
- To evaluate an algorithm or to compare two algorithms, we focus on their relative rates of growth wrt the increase of the input size.
- The average running time is difficult to determine.
- We focus on the worst case running time
  - Easier to analyze
  - Crucial to applications such as finance, robotics, and games





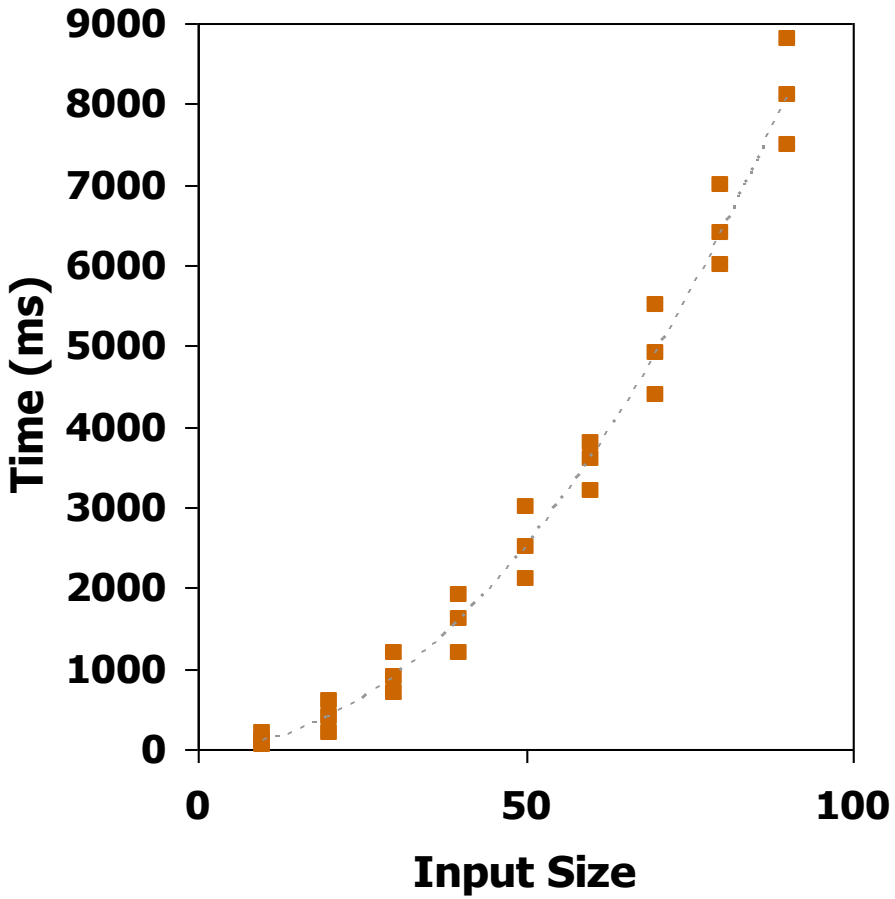
Running Time

- Problem: prefix averages
  - Given an array X
  - Compute the array A such that A[i] is the average of elements X[0] … X[i], for i=0..n-1
- Sol 1
  - At each step i, compute the element X[i] by traversing the array A and determining the *sum* of its elements, respectively the average
- Sol 2
  - At each step i update a *sum* of the elements in the array A
  - Compute the element X[i] as *sum/I*

Big question: Which solution to choose?

Experimental Approach

- Write a program to implement the algorithm.
- Run this program with inputs of varying size and composition.
- Get an accurate measure of the actual running time (e.g. system call date).
- Plot the results.
- Problems?





Limitations of Experimental Studies

- The algorithm has to be implemented, which may take a long time and could be very difficult.
- Results may not be indicative for the running time on other inputs that are not included in the experiments.
- In order to compare two algorithms, the same hardware and software must be used.



Use a Theoretical Approach

- Based on high-level description of the algorithms, rather than language dependent implementations
- Makes possible an evaluation of the algorithms that is independent of the hardware and software environments

  → **Generality**



Pseudocode

- High-level description of an algorithm.
- More structured than plain English.
- Less detailed than a program.
- Preferred notation for describing algorithms.
- Hides program design issues.

Example: find the max element of an array

```
Algorithm arrayMax(A, n)
  Input array A of n integers
  Output maximum element of A

  currentMax ← A[0]
  for i ← 1 to n – 1 do
    if A[i] > currentMax then
      currentMax ← A[i]
  return currentMax
```



Pseudocode

- Control flow
  - **if** … **then** … [**else** …]
  - **while** … **do** …
  - **repeat** … **until** …
  - **for** … **do** …
  - Indentation replaces braces
- Method declaration

  Algorithm ***method*** (***arg*** [, ***arg***…])

  Input …

  Output …
- Method call

  var.method (***arg*** [, ***arg***…])
- Return value

  return ***expression***
- Expressions

  ← Assignment (equivalent to =)

  = Equality testing (equivalent to ==)

  n^2 Superscripts and other mathematical formatting allowed



Primitive Operations

- The basic computations performed by an algorithm
- Identifiable in pseudocode
- Largely independent from the programming language
- Exact definition not important
- Use comments
- Instructions have to be basic enough and feasible!

- Examples:
  - Evaluating an expression
  - Assigning a value to a variable
  - Calling a method
  - Returning from a method



Low Level Algorithm Analysis

- Based on primitive operations (low-level computations independent from the programming language)
- E.g.:
  - Make an addition = 1 operation
  - Calling a method or returning from a method = 1 operation
  - Index in an array = 1 operation
  - Comparison = 1 operation etc.
- Method: Inspect the pseudo-code and count the number of primitive operations executed by the algorithm



Counting Primitive Operations

- By inspecting the code, we can determine the number of primitive operations executed by an algorithm, as a function of the input size.

Algorithm *arrayMax*(*A*, *n*)	# operations
currentMax ← *A*[0]	2
for *i* ← 1 **to** *n* – 1 **do**	2 + ***n***
if *A*[*i*] > *currentMax* **then**	2(***n*** – 1)
currentMax ← *A*[*i*]	2(***n*** – 1)
{ increment counter ***i*** }	2(***n*** – 1)
return *currentMax*	1
Total	7***n*** – 1



Estimating Running Time

- Algorithm ***arrayMax*** executes $7n - 1$ primitive operations.
- Let's define

  a:= Time taken by the fastest primitive operation

  b:= Time taken by the slowest primitive operation

- Let $T(n)$ be the actual running time of ***arrayMax***. We have

$$a\,(7n - 1) \leq T(n) \leq b(7n - 1)$$

- Therefore, the running time $T(n)$ is bounded by two linear functions.

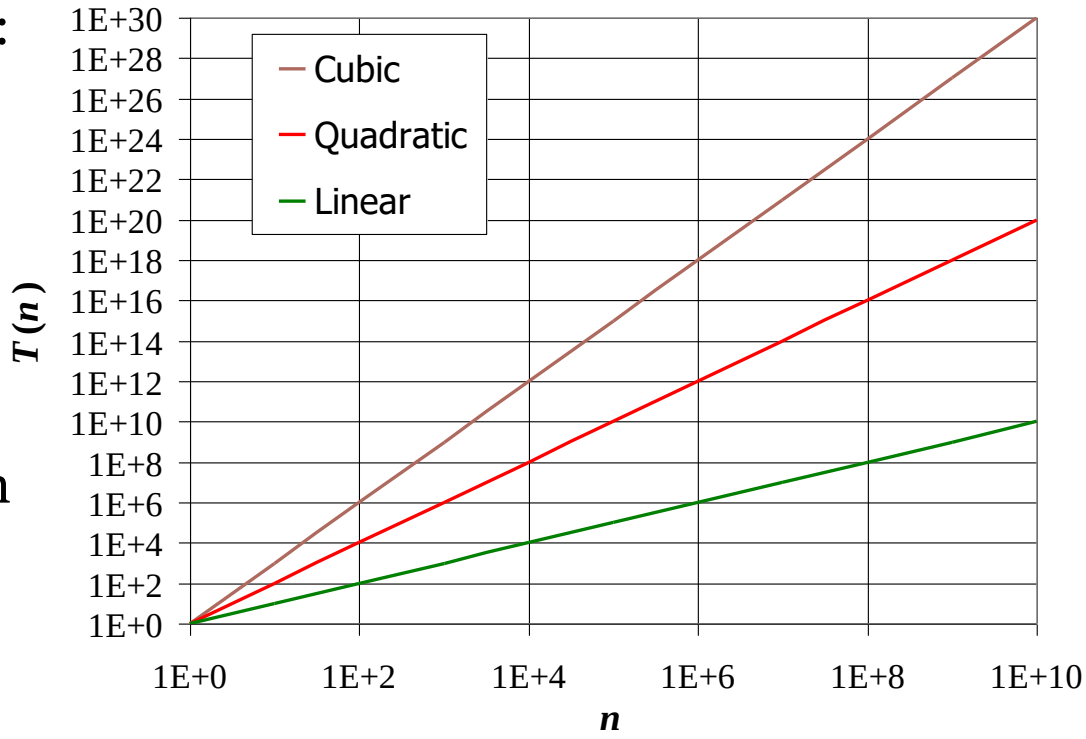


Growth Rate of Running Time

- Changing computer hardware / software
  - Affects $T(n)$ by a constant factor
  - Does not alter the growth rate of $T(n)$
- The linear growth rate of the running time $T(n)$ is an intrinsic property of algorithm ***arrayMax***

Growth Rates

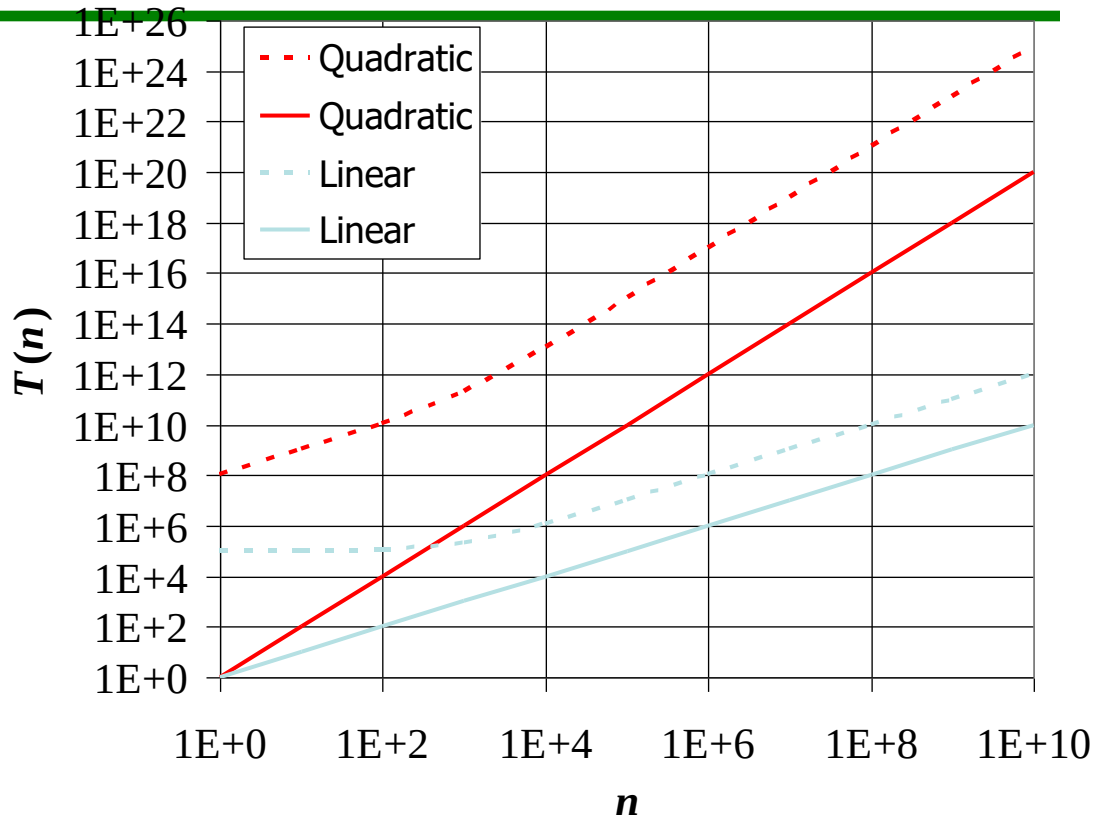
- Growth rates of functions:
  - Linear ≈ n
  - Quadratic ≈ n^2
  - Cubic ≈ n^3
- In a log-log chart, the slope of the line corresponds to the growth rate of the function





Constant Factors

- The growth rate is not affected by
  - constant factors or
  - lower-order terms
- Examples
  - $10^2n + 10^5$ is a linear function
  - $10^5n^2 + 10^8n$ is a quadratic function





Asymptotic Notation

- Need to abstract further
- Give an “idea” of how the algorithm performs
- n steps vs. n+5 steps
- n steps vs. n^2 steps



Problem

- Fibonacci numbers
  - F[0] = 0
  - F[1] = 1
  - F[i] = F[i-1] + F[i-2] for i ≥ 2
- Pseudo-code
- Number of operations